# NeuroShard: A Decentralized Architecture for Collaborative Large Language Model Training over Heterogeneous Networks

Linir Zamir
*Florida Atlantic University*
Boca Raton, FL, USA
lzamir2016@fau.edu

***Abstract*—Training large language models (LLMs) currently requires tightly integrated, billion-dollar data-center clusters controlled by a handful of organizations. We present NeuroShard, a fully decentralized protocol that enables collaborative LLM training across heterogeneous commodity hardware over wide-area peer-to-peer (P2P) networks. NeuroShard introduces three key mechanisms: (1) *Quorum-Based Training*, where speed-matched nodes self-organize into complete training pipelines that synchronize internally while coordinating across quorums via DiLoCo-style gradient aggregation, reducing inter-node communication by over 500× compared to synchronous data parallelism; (2) *Proof of Neural Work (PoNW)*, an optimistic verification consensus protocol in which cryptographically signed training proofs are accepted immediately and subject to a stake-weighted challenge window, achieving high throughput without sacrificing accountability; and (3) *Byzantine-robust gradient aggregation* using $\sqrt{n}$-weighted contributions with Trimmed Mean and Coordinate-wise Median defenses that tolerate up to 30% adversarial participants. The architecture supports multiple contribution modes—pipeline, asynchronous, data provision, and verification—enabling devices ranging from data-center GPUs to single-board computers to participate meaningfully. We present the system design, formal protocol descriptions, and a security analysis demonstrating NeuroShard's resilience to gradient poisoning, Sybil, and free-riding attacks.**

***Index Terms*—Decentralized machine learning, distributed training, peer-to-peer networks, Byzantine fault tolerance, consensus protocols, large language models**

## I. Introduction

The training of frontier large language models (LLMs) such as GPT-4 [1] and LLaMA [2] is concentrated within a small number of vertically integrated organizations that control the models, data pipelines, and distribution channels. This centralization creates three systemic problems. First, alignment and safety policies are set unilaterally with limited transparency. Second, the infrastructure cost of tightly coupled GPU clusters—often exceeding $100M per training run—excludes independent researchers and smaller organizations. Third, the world's vast reservoir of edge compute (consumer GPUs, idle CPUs, mobile devices) remains unused for AI workloads.

Meanwhile, distributed and federated learning research has demonstrated that gradient-based optimization can be decomposed across geographically dispersed nodes [3], [4]. However, existing approaches face fundamental limitations when applied to open, adversarial networks: Federated Learning (FL) assumes a trusted central aggregator [3]; pipeline-parallel systems like Petals [5] target inference rather than training; and volunteer computing frameworks such as Hivemind [6] lack robust verification and incentive mechanisms.

NeuroShard addresses these gaps with an integrated protocol for *trustless, incentivized, Byzantine-tolerant* collaborative training. The key insight is that a heterogeneous P2P network can be partitioned into speed-matched *quorums*—self-organized groups that form complete training pipelines—which train synchronously within the group and synchronize across groups only periodically via DiLoCo [7] outer-loop updates. This two-level synchronization hierarchy reduces communication bandwidth by over 500× while tolerating node heterogeneity, churn, and adversarial behavior.

**Contributions.** This paper makes the following contributions:

- **Quorum-Based Training:** A self-organization protocol where speed-matched nodes form complete pipelines, enabling heterogeneous hardware (from H100 servers to Raspberry Pis) to contribute through appropriate contribution modes (Section IV).
- **DiLoCo Cross-Quorum Synchronization:** An adaptation of Distributed Low-Communication learning with Nesterov momentum outer optimization, achieving 500× bandwidth reduction over synchronous methods (Section V).
- **Proof of Neural Work (PoNW):** An optimistic verification consensus mechanism with ECDSA-signed proofs, challenge-response fraud detection, and stake-based slashing that makes cheating provably unprofitable (Section VI).
- **Byzantine-Robust Aggregation:** $\sqrt{n}$-weighted gradient aggregation with Trimmed Mean and Coordinate-wise Median defenses, tolerating up to 30% adversarial nodes (Section VII).
- **Security Analysis:** Formal analysis of resistance to gradient poisoning, Sybil, free-riding, and quorum collusion attacks (Section VIII).

## II. Related Work

### A. Federated and Distributed Learning

Federated Learning (FL) [3] enables training across decentralized devices by keeping data local and aggregating model updates at a central server. FedAvg [3] and its successors [19], [20] address statistical heterogeneity but fundamentally rely on a trusted aggregator. NeuroShard eliminates this single point of failure through gossip-based gradient sharing and Byzantine-robust aggregation. Data parallelism with AllReduce [21] requires synchronous communication every training step, which is infeasible over wide-area networks with heterogeneous bandwidth. Local SGD methods [22], [23] reduce communication frequency but still assume homogeneous participants.

### B. Pipeline Parallelism and Decentralized Inference

GPipe [24] and PipeDream [25] partition models by layers across devices for pipeline-parallel training, but assume reliable, high-bandwidth interconnects within a data center. Petals [5] extends pipeline parallelism to volunteer networks for *inference*, demonstrating that geographically distributed nodes can serve a shared model. NeuroShard goes further by enabling distributed *training* with cryptographic verification of contributions.

### C. Decentralized AI Networks

Bittensor [17] creates a decentralized intelligence market where models are evaluated by output quality, but treats model training as a black box. Gensyn [18] focuses on verifiable ML compute but lacks a complete training coordination protocol. Hivemind [6] provides a framework for collaborative training but offers limited Byzantine resilience and no economic incentives. NeuroShard uniquely combines a custom model architecture designed for decentralized training, gradient-level verification, robust aggregation, and token-based incentive alignment.

### D. DiLoCo and Low-Communication Training

DiLoCo [7] demonstrates that independent replicas can train with infrequent synchronization by exchanging pseudo-gradients and applying an outer optimizer with Nesterov momentum. NeuroShard adapts DiLoCo to an adversarial, heterogeneous setting by replacing trusted averaging with Byzantine-robust aggregation and adding $\sqrt{n}$-weighted contributions for fair influence.

## III. System Architecture

### A. Network Layer

NeuroShard operates on a structured P2P overlay built on a Kademlia Distributed Hash Table (DHT) [8]. Nodes discover peers through a hybrid mechanism: initial bootstrap via tracker servers, followed by fully decentralized DHT-based discovery and a gossip protocol for redundancy. Each node advertises its capabilities to the DHT: available memory, compute speed (measured via self-benchmarking), network bandwidth, currently held model layers, quorum membership, staked tokens, and historical reputation.

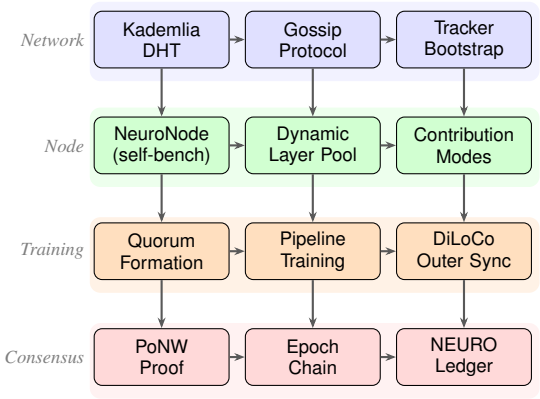


Fig. 1. NeuroShard layered architecture. The network layer provides decentralized discovery; nodes self-benchmark and claim layer assignments; quorums execute pipeline training with DiLoCo outer-loop synchronization; and PoNW proofs are chained into epochs to drive the NEURO reward ledger.

TABLE I
Speed Tier Classification

| Tier | Latency/Layer | Hardware | Role |
|---|---|---|---|
| T1 | <10 ms | H100, A100 | Pipeline leader |
| T2 | 10–50 ms | RTX 4090/3090 | Pipeline member |
| T3 | 50–200 ms | RTX 3060, fast CPU | Pipeline member |
| T4 | 200–1000 ms | Older GPU, CPU | Pipeline/async |
| T5 | >1000 ms | Raspberry Pi | Async only |

### B. Speed Tiers and Contribution Modes

Upon joining, every node self-benchmarks by measuring forward-pass latency per transformer layer and is classified into one of five speed tiers:

Nodes operate in one of six contribution modes based on their tier and network needs: (1) *Pipeline*: real-time quorum member processing activations synchronously; (2) *Async*: offline training with periodic gradient submission; (3) *Data*: storing and serving training data shards; (4) *Verify*: monitoring and challenging PoNW proofs; (5) *Inference*: serving inference requests; (6) *Idle*: available but inactive.

### C. Dynamic Model Architecture

NeuroShard trains NeuroLLM, a transformer with RMSNorm [11], Rotary Position Embeddings (RoPE) [12], Grouped Query Attention [13], SwiGLU activations [14], and sparse Mixture-of-Experts (MoE) layers with top-2 routing. Crucially, the model architecture is not fixed: both depth and width scale automatically with total network capacity following empirical scaling laws [9], [10]:

$$\text{Width (hidden_dim)} \propto M_{\text{total}}^{0.6}, \tag{1}$$

$$\text{Depth (num_layers)} \propto M_{\text{total}}^{0.4}, \tag{2}$$

where $M_{\text{total}} = \sum_{i=1}^{N} M_i$ is the aggregate memory across all $N$ participating nodes. This ensures the model grows organically—from 50M parameters with a single 2 GB node to over 120B parameters with 1000 nodes averaging 8 GB each—without manual intervention or synchronized upgrade phases.

Layers are distributed across nodes via a DHT-based *Dynamic Layer Pool*. Each layer maintains a minimum replication factor $R \geq 2$ for fault tolerance, and scarcity-based incentives encourage nodes to host under-replicated layers:

$$M_{\text{scarcity}} = 1 + 0.5 \times \frac{R_{\text{target}} - R_{\text{actual}}}{R_{\text{target}}}. \quad (3)$$

## IV. Quorum-Based Training

### A. Quorum Formation

A *quorum* is a self-organized group of speed-compatible nodes that collectively hold all model layers, forming a complete training pipeline. Quorum formation proceeds as follows:

1) An initiating node queries the DHT for layer holders in compatible speed tiers.
2) A greedy selection algorithm maximizes layer coverage while minimizing inter-node latency.
3) If complete coverage is achieved, the initiator proposes the quorum to all selected members.
4) Upon unanimous acceptance, the quorum is registered in the DHT.
5) If coverage is incomplete, the node falls back to asynchronous mode.

Quorums have four key properties: *completeness* (all layers from embedding to LM head are covered), *speed matching* (members drawn from compatible tiers, e.g., T2–T3), *self-sufficiency* (no external coordinator required), and *temporality* (sessions last approximately one hour, with renewal at 80% of the session if health checks pass).

### B. Pipeline Execution

Within a quorum, nodes assume roles based on their layer assignments:

- **Initiator** (embedding layer): Fetches a training batch from Genesis data providers, computes token embeddings, and begins the forward pass.
- **Processors** (middle layers): Receive activations from the preceding node, compute their assigned layers, and forward to the next.
- **Finisher** (LM head): Computes cross-entropy loss against ground-truth labels and initiates the backward pass, propagating gradients back through the pipeline.

Each node applies local AdamW optimization to its assigned layers after receiving gradients, accumulating updates for the next cross-quorum synchronization round.

### C. Multi-Speed Quorums

The network naturally forms multiple quorums operating at different throughputs: fast quorums (T1–T2, $\sim$16 batches/s), medium quorums (T2–T3, $\sim$3 batches/s), and slow quorums (T3–T4, $\sim$0.5 batches/s). Each quorum is internally synchronized but operates asynchronously relative to other quorums. Additionally, T5 nodes that cannot meet real-time pipeline requirements contribute via asynchronous mode: they download current weights, train locally, compute pseudo-gradients $\Delta w = w_{\text{initial}} - w_{\text{final}}$, and submit them to the next synchronization round. Async contributions are weighted by a freshness decay function to prioritize recent work.

TABLE II
Communication Cost: Synchronous vs. DiLoCo ($H$=500)

| Metric | Synchronous | DiLoCo |
|---|---|---|
| Syncs per 1000 steps | 1000 | 2 |
| Relative bandwidth/step | 100% | 0.2% |
| Straggler tolerance | Low | High |

### D. Gradient Compression

To minimize bandwidth within and across quorums, NeuroShard applies a multi-stage compression pipeline: (1) Top-$K$ sparsification retaining only the largest-magnitude gradient elements; (2) INT8 quantization of selected values; (3) entropy coding via zlib; and (4) error feedback, where residuals from discarded values are accumulated locally and added to the next step's gradient to preserve convergence. This pipeline achieves 50–100$\times$ compression with minimal impact on convergence quality.

## V. DiLoCo Cross-Quorum Synchronization

### A. Two-Level Synchronization Hierarchy

NeuroShard's communication architecture operates at two levels. The *inner loop* consists of standard synchronous pipeline training within each quorum, with no cross-quorum communication. After every $H$ inner steps (default $H$=500), the *outer loop* triggers cross-quorum synchronization for each model layer.

### B. Outer Loop Protocol

At each outer synchronization point, all holders of a given layer form a *Layer Cohort* and execute the following protocol:

1) Each node computes its pseudo-gradient: $\Delta w = w_0 - w_H$, where $w_0$ are the weights at the start of the inner loop and $w_H$ are the weights after $H$ local steps.
2) Pseudo-gradients are exchanged within the Layer Cohort via gossip (each node contacts $k$=3 random cohort peers).
3) The aggregated pseudo-gradient is computed using Byzantine-robust aggregation (Section VII).
4) The outer optimizer applies the aggregated update with Nesterov momentum:

$$m_{t+1} = \beta \cdot m_t + \Delta w_{\text{agg}}, \quad (4)$$

$$w_{t+1} = w_t + \eta_{\text{outer}} \cdot (\beta \cdot m_{t+1} + \Delta w_{\text{agg}}), \quad (5)$$

where $\beta = 0.9$ and $\eta_{\text{outer}} = 0.7$.

### C. Bandwidth Analysis

Table II compares the communication cost of standard synchronous training (gradient exchange every step) with NeuroShard's DiLoCo approach.

With gradient compression applied to the pseudo-gradients, the effective bandwidth reduction exceeds 500$\times$ per training

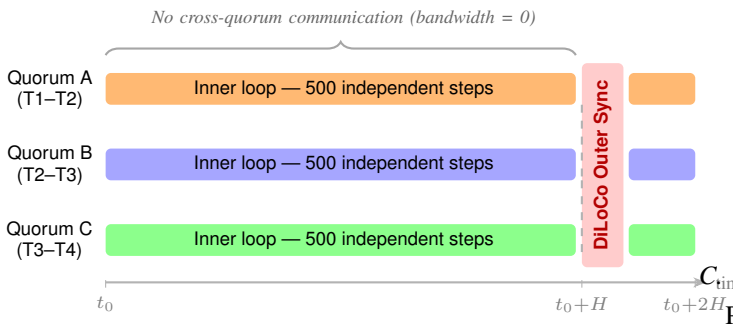


Fig. 2. Two-level synchronization hierarchy. Three quorums (fast, medium, slow) train independently for $H$=500 inner steps with zero cross-quorum bandwidth. At $t_0+H$, a DiLoCo outer-sync aggregates pseudo-gradients from all Layer Cohorts via Byzantine-robust aggregation and Nesterov momentum, then each quorum resumes its next inner loop.

step relative to AllReduce-based synchronous training, making wide-area P2P training practical even over consumer internet connections.

## VI. Proof of Neural Work

### A. Design Rationale

In a trustless decentralized network, nodes must prove that they have performed genuine training computation to receive rewards. Traditional consensus mechanisms are ill-suited: Proof of Work (PoW) wastes energy on arbitrary hash computation, while Proof of Stake (PoS) requires capital lockup without guaranteeing useful work. NeuroShard introduces *Proof of Neural Work* (PoNW), which leverages the inherent verifiability of neural network training—a gradient computed on real data will reduce the model's loss when aggregated, whereas fake or random gradients will not.

### B. Optimistic Verification Protocol

PoNW employs an *optimistic* verification model inspired by optimistic rollups [16]: proofs are accepted immediately upon submission and become final after a challenge window expires without dispute. The protocol operates as follows:

1) **Proof Submission:** Upon completing a training round, a node constructs a proof containing: node identity, work metrics (batches processed, tokens generated, layers held), model state hashes before and after training ($h_{\text{start}}, h_{\text{end}}$ with $h_{\text{start}} \neq h_{\text{end}}$), a gradient commitment $c = \text{SHA-256}(h_{\text{model}} \| h_{\text{data}} \| \|\nabla\|_2 \| \mathcal{L})$, and an epoch identifier $e = \lfloor t_{\text{unix}}/60 \rfloor$. The proof is signed using ECDSA over secp256k1 and submitted to the DHT.
2) **Challenge Window:** A 10-minute window opens during which any staked node may challenge the proof by posting a 10 NEURO bond.
3) **Resolution:** If unchallenged, the proof is finalized and rewards are distributed. If challenged, the prover must supply verification data (activations, gradient samples) for the challenger to recompute. Fraud triggers 2× stake slashing of the prover; false challenges forfeit the challenger's bond.

TABLE III
Optimistic Verification Payoff Matrix

| Outcome | Prover | Challenger |
|---|---|---|
| No challenge | $+R$ | N/A |
| Valid proof challenged | $+R+0.5B$ | $-B$ |
| Fraud detected | $-2S$ | $+S$ |

### C. Chained Proof Structure

PoNW proofs are organized into a chain analogous to a blockchain. Each *epoch* (one Unix minute) aggregates all proofs submitted within its window into a Merkle tree, linked to the previous epoch via cryptographic hash:

$$\text{epoch}_i.\text{prev_hash} = \text{Hash}(\text{epoch}_{i-1}). \tag{6}$$

Model state commitments enforce training continuity: $\text{epoch}_i.\text{model_start} = \text{epoch}_{i-1}.\text{model_end}$. Each epoch is signed by a stake-weighted proposer selected deterministically. This chain provides immutability, auditability, and global consensus on training progress without a coordinator.

### D. Game-Theoretic Analysis

Table III summarizes the payoff structure. Let $R$ denote the proof reward, $S$ the prover's stake, $p$ the probability of challenge, and $B$ the challenger's bond.

For a rational adversary, submitting a fraudulent proof has expected value $\mathbb{E}[\text{fraud}] = (1-p) \cdot R - p \cdot 2S$. With adaptive verification rates ($p$ scales from 1% at 20 nodes to 5% at 500+ nodes) and $S \gg R$, the expected value is strictly negative, making honest participation the dominant strategy.

### E. Adaptive Verification Rate

The spot-check probability scales with network size to balance security against verification overhead:

$$P_{\text{verify}} = \begin{cases} 0 & \text{if } N < 20, \\ 0.01 & \text{if } 20 \leq N < 100, \\ 0.03 & \text{if } 100 \leq N < 500, \\ 0.05 & \text{if } N \geq 500. \end{cases} \tag{7}$$

## VII. Byzantine-Robust Gradient Aggregation

### A. Threat Model

We assume up to $f < 0.3N$ nodes in any Layer Cohort may be adversarial, submitting arbitrary gradient vectors to degrade model quality or inject backdoors. Adversaries may coordinate but cannot break ECDSA signatures.

### B. Aggregation Methods

NeuroShard implements three robust aggregation strategies, selectable per cohort based on network conditions:

**Trimmed Mean.** For each coordinate $j$ of the gradient vector, the top and bottom $k\%$ of submitted values are discarded before averaging:

$$\bar{g}_j = \frac{1}{N - 2\lfloor kN \rfloor} \sum_{i \in \mathcal{S}_j} g_{i,j}, \tag{8}$$

TABLE IV
ATTACK TYPES AND DEFENSES

| Attack | Detection | Defense | Penalty |
|---|---|---|---|
| Free-riding | PoNW spot-checks | Gradient commitment verification | $2\times$ slash |
| Gradient poison | Magnitude & distribution checks | Robust aggregation | Gradient rejected |
| Sybil | Stake analysis | Min. stake requirement | All clones slashed |
| Quorum collusion | Cross-quorum audit | Random verification | Quorum dissolved |
| Replay | Timestamp & nonce checks | 5-min freshness window | Proof rejected |

where $\mathcal{S}_j$ is the set of non-trimmed indices for coordinate $j$. This neutralizes outlier gradients introduced by adversaries.

**Coordinate-wise Median.** The median value is taken for each coordinate independently:

$$\bar{g}_j = \text{median}(\{g_{1,j}, g_{2,j}, \ldots, g_{N,j}\}). \tag{9}$$

This theoretically tolerates up to 50% adversarial participants per coordinate.

**Multi-Krum [15].** Selects the $m$ gradient vectors closest to the spatial center of the distribution by minimizing pairwise distances, then averages the selected subset. This provides whole-vector robustness against coordinated attacks.

### C. $\sqrt{n}$-Weighted Contributions

To prevent large nodes from dominating the aggregation, contributions are weighted by the square root of batches processed rather than the raw count:

$$w_i = \sqrt{B_i} \times f_i, \tag{10}$$

where $B_i$ is the batch count and $f_i$ is a freshness decay factor. A node processing 100 batches thus has $10\times$ the influence of a node processing 1 batch, not $100\times$. This ensures fair representation while still rewarding greater contribution.

## VIII. SECURITY ANALYSIS

### A. Attack Resistance

Table IV summarizes the attack surface and corresponding defenses.

### B. Gradient Poisoning Resilience

Valid gradients exhibit predictable statistical properties: approximately Gaussian distribution, magnitude within expected ranges for the batch size, and consistent sparsity patterns after Top-$K$ compression. NeuroShard applies three layers of defense: (1) statistical screening rejects gradients deviating more than $3\sigma$ from the cohort mean; (2) robust aggregation (Trimmed Mean or Median) neutralizes remaining outliers; and (3) loss-based verification confirms that the aggregated update reduces validation loss, i.e., $\mathcal{L}(w - \eta g_{\text{agg}}) < \mathcal{L}(w)$.

### C. Sybil Resistance

Creating multiple identities to amplify influence is mitigated by: (a) a minimum staking requirement of 10 NEURO for verification roles, making Sybil attacks economically costly; (b) $\sqrt{n}$-weighting, which yields sublinear returns for splitting work across identities; and (c) IP and hardware fingerprinting heuristics that flag suspicious clusters.

### D. Free-Riding Prevention

Nodes cannot earn rewards without performing verifiable computation. Each PoNW proof includes a gradient commitment $c = \text{SHA-256}(h_{\text{model}} \| h_{\text{data}} \| \|\nabla\|_2 \| \mathcal{L})$ that can be spot-checked by recomputing the training step on the same data shard. The deterministic data sharding scheme (shard $=$ Hash(node_id) $\bmod S$) enables any verifier to independently obtain the same training batch and validate the commitment.

## IX. INCENTIVE MECHANISM

Training rewards form the dominant component of the NEURO token economy. A node's reward per proof epoch is:

$$R = R_{\text{base}} \times L \times B \times M_{\text{role}} \times M_{\text{scarcity}} \times M_{\text{stake}}, \tag{11}$$

where $R_{\text{base}} = 0.0005$ NEURO per batch per layer, $L$ is the number of layers held, $B$ is the batch count, $M_{\text{role}} \in \{1.0, 1.2, 1.3\}$ for processor, initiator, and finisher roles respectively, $M_{\text{scarcity}}$ is defined in (3), and

$$M_{\text{stake}} = 1 + 0.1 \cdot \log_2\left(1 + \frac{S}{1000}\right) \tag{12}$$

provides logarithmically diminishing returns on staked capital, preventing wealth concentration while still incentivizing commitment. Under this formula, a node staking 100,000 NEURO receives only $1.66\times$ rewards versus $11\times$ under a linear model.

A 5% burn on all token transfers and inference payments creates deflationary pressure, while 50% of slashed stakes are permanently burned, aligning long-term token value with network utility.

## X. DISCUSSION

### A. Scalability

NeuroShard's two-level synchronization hierarchy decouples intra-quorum communication (high-frequency, low-latency within speed-matched groups) from inter-quorum communication (low-frequency DiLoCo outer steps). This separation means the system scales with the number of quorums rather than individual nodes: adding more quorums increases aggregate training throughput linearly without increasing per-node communication overhead. The dynamic model architecture further ensures that increased participation translates directly into a more capable model.

### B. Convergence Properties

DiLoCo has been shown to converge comparably to fully synchronous training when the inner loop length $H$ is moderate [7]. NeuroShard's use of Nesterov momentum in the outer optimizer (Equations 4–5) accelerates convergence, while Byzantine-robust aggregation ensures that adversarial contributions do not derail the optimization trajectory. The $\sqrt{n}$-weighting scheme provides a balance between leveraging high-contribution nodes and maintaining democratic influence.

### C. Limitations and Future Work

The current design has several limitations. First, quorum formation assumes sufficient node diversity; in bootstrapping scenarios with few participants, pipeline coverage may be incomplete. Second, the optimistic verification model introduces a latency floor equal to the challenge window (10 minutes) before rewards are finalized. Third, formal convergence guarantees under Byzantine-robust aggregation with heterogeneous data distributions remain an open theoretical question. Future work will address these through adaptive challenge windows, formal convergence proofs under non-i.i.d. data, and integration of trusted execution environments (TEEs) for hardware-attested verification.

## XI. Conclusion

We have presented NeuroShard, a decentralized protocol for collaborative LLM training that transforms a heterogeneous P2P network into a unified training system. Through Quorum-Based Training, nodes self-organize into speed-matched pipelines that operate independently while synchronizing via DiLoCo outer-loop updates, achieving over 500× bandwidth reduction. Proof of Neural Work provides trustless verification with optimistic throughput and economic security through stake-based challenges. Byzantine-robust aggregation with $\sqrt{n}$-weighted contributions ensures resilience against adversarial participants. Together, these mechanisms demonstrate that community-driven, decentralized LLM training is architecturally feasible—offering a path toward AI systems that are collectively owned, transparently governed, and open to all.

## References


[1] OpenAI, "GPT-4 technical report," *arXiv:2303.08774 [cs.CL]*, Mar. 2023.

[2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv:2302.13971 [cs.CL]*, Feb. 2023.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, Fort Lauderdale, FL, 2017, pp. 1273–1282.

[4] P. Kairouz *et al.*, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.

[5] A. Borzunov, D. Baranchuk, T. Dettmers, M. Riabinin, Y. Belkada, A. Chumachenko, P. Samygin, and C. Raffel, "Petals: Collaborative inference and fine-tuning of large models," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguist. (ACL): System Demonstrations*, Toronto, Canada, 2023, pp. 38–44.

[6] M. Ryabinin and A. Gusev, "Towards crowdsourced training of large neural networks using decentralized mixture-of-experts," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 3659–3672.

[7] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen, "DiLoCo: Distributed low-communication training of language models," *arXiv:2311.08105 [cs.LG]*, Nov. 2023.

[8] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in *Proc. 1st Int. Workshop Peer-to-Peer Syst. (IPTPS)*, Cambridge, MA, Mar. 2002, pp. 53–65.

[9] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *arXiv:2001.08361 [cs.LG]*, Jan. 2020.

[10] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 30016–30030.

[11] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 12360–12371.

[12] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv:2104.09864 [cs.CL]*, Apr. 2021.

[13] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, 2023, pp. 4895–4901.

[14] N. Shazeer, "GLU variants improve transformer," *arXiv:2002.05202 [cs.LG]*, Feb. 2020.

[15] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 118–128.

[16] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in *Proc. 27th USENIX Security Symp.*, Baltimore, MD, Aug. 2018, pp. 1353–1370.

[17] Y. Const and J. Fishman, "Bittensor: A peer-to-peer intelligence market," *Bittensor Whitepaper*, 2022. [Online]. Available: https://bittensor.com/whitepaper

[18] Gensyn, "Gensyn technical paper," 2023. [Online]. Available: https://docs.gensyn.ai

[19] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, vol. 119, 2020, pp. 5132–5143.

[20] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. 3rd Conf. Mach. Learn. Syst. (MLSys)*, Austin, TX, 2020.

[21] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch SGD: Training ImageNet in 1 hour," *arXiv:1706.02677 [cs.CV]*, Jun. 2017.

[22] S. U. Stich, "Local SGD converges fast and communicates little," in *Proc. 7th Int. Conf. Learn. Represent. (ICLR)*, New Orleans, LA, 2019.

[23] T. Lin, S. U. Stich, K. K. Patel, and M. Jaggi, "Don't use large mini-batches, use local SGD," in *Proc. 8th Int. Conf. Learn. Represent. (ICLR)*, Addis Ababa, Ethiopia, 2020.

[24] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 103–112.

[25] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "PipeDream: Generalized pipeline parallelism for DNN training," in *Proc. 27th ACM Symp. Oper. Syst. Princ. (SOSP)*, Huntsville, ON, Canada, 2019, pp. 1–15.